

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 08 -

PROJETO FINAL: DESENVOLVENDO E PUBLICANDO UMA API

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	39
REFERÊNCIAS.....	40

EMSE

O QUE VEM POR AÍ?

Nesta aula, vamos colocar em produção a segunda versão da nossa API, agora com todas as soluções que implementamos nas aulas anteriores como rate limiting, logs estruturados e métricas com Prometheus e Grafana, além da inclusão de um novo endpoint para processamento em batch das previsões.

Ao final da aula você terá uma API pronta e funcional, que será um bom ponto de partida para seus futuros projetos.

HANDS ON

Prepare-se para mais uma aula hands on, onde vamos fazer o deploy da nossa API completa no render!

EMSE

SAIBA MAIS

Do MVP à Produção

A API (desenvolvida nas aulas anteriores) que estamos utilizando já está equipada com as seguintes funcionalidades cruciais:

- **Autenticação JWT:** acesso seguro através do endpoint POST /login.
- **Classificação de Flores:** serviço principal no endpoint POST /predict.
- **Verificação de Saúde (Health Check):** disponível em GET /health para monitoramento de status.
- **Documentação Interativa:** acesso ao Swagger em GET /docs.
- **Pronta para Contêineres:** inclui um Dockerfile para fácil implantação em ambientes containerizados.

Para que a API esteja apta para um ambiente de produção robusto, vamos implementar alguns desafios a mais:

DESAFIO	SOLUÇÃO NECESSÁRIA	STATUS
Prevenção de uso excessivo (abuso)	Implementação de Rate Limiting	×
Melhoria na análise e depuração de erros	Adoção de Logs Estruturados (formato JSON)	×
Otimização do processamento em massa	Desenvolvimento de funcionalidade de Batch Prediction	×
Monitoramento de desempenho e comportamento	Integração de Métricas com Prometheus	×
Criação de um ambiente de monitoramento em produção	Configuração dos webservices no render via blueprint (render.yaml)	×

Tabela 1 - Desafios
Fonte: Elaborado pelo autor (2026)

Ao término deste capítulo, você terá em funcionamento duas versões distintas da API:

- **api-iris.onrender.com** (v1 - Versão base, desenvolvida na aula 6).
- **https://api-iris-v2-d20g.onrender.com/** (v2 - Versão completa, desenvolvida nesta aula).

Ps: o endereço da API vai mudar de acordo com o que você escolher ao configurar o Render.

Desafio 1 - Rate Limiting

O Rate Limiting é uma defesa essencial para garantir a estabilidade e disponibilidade da sua API. Veja por que:

Cenário 1: Ataque de Negação de Serviço Distribuído (DDoS)

- **Fluxo de Eventos:**
 - Atacante envia 10.000 req/seg.
 - API tenta processar todas as requisições.
 - Memória e CPU são sobrecarregadas.
- **Resultado:**
 - Serviço indisponível para TODOS os usuários.

Cenário 2: Cliente com Comportamento Inesperado (Bug)

- **Fluxo de Eventos:**
 - App mobile entra em loop infinito.
 - Envia 1000 requisições por segundo ininterruptamente.
- **Resultado:**
 - Consome toda a largura de banda disponível.
 - Outros usuários experimentam lentidão

A Solução: Rate Limiting (Exemplo com slowapi)

O Rate Limiting protege o serviço, bloqueando o tráfego excessivo:

USUÁRIO	TAXA DE REQUISIÇÕES	RESULTADO
Usuário A	30 req/min	 OK
Usuário B	50 req/min	 "429 Too Many Requests" (Limite excedido)
Atacante	10.000 req/min	 Bloqueado após atingir 30

Tabela 2 - Rate Limiting
Fonte: Elaborado pelo autor (2026)

O controle de taxa pode ser aplicado de diversas maneiras:

ESTRATÉGIA	DESCRIÇÃO	EXEMPLO
Por IP	Limite baseado no endereço IP de origem.	60 requisições por minuto por IP.
Por Usuário	Limite aplicado a usuários autenticados (mais granular).	100 requisições por minuto por usuário.
Por Endpoint	Limites específicos para rotas distintas, conforme a carga.	/predict: 30/min; /login: 10/min.
Global	Limite total para toda a API, protegendo a capacidade máxima do servidor.	10.000 requisições por minuto total.

STabela 3 – Maneiras de aplicar o controle de taxa

Fonte: Elaborado pelo autor (2026)

O algoritmo Token Bucket é um mecanismo de limitação de taxa de requisições (Rate Limiting) muito comum, sendo utilizado, por exemplo, pela biblioteca slowapi.

COMPONENTE	DESCRIÇÃO
Balde	Possui uma Capacidade Máxima (e.g., 30 tokens).
Recarga	A taxa de Recarga define a frequência com que novos tokens são adicionados (e.g., 1 token a cada 2 segundos).
Consumo	Cada requisição HTTP consome 1 token.
Ação	Se o balde estiver vazio, a requisição é rejeitada com um Erro 429 (Too Many Requests).

Tabela 4 - Como o Token Bucket funciona (exemplo do slowapi)
Fonte: Elaborado pelo autor (2026)

A seguir, temos um exemplo de como configurar e aplicar o Rate Limiting usando a biblioteca slowapi.

Importações Necessárias:

```
from slowapi import Limiter
from slowapi.util import get_remote_address # Para obter o IP do cliente
```

Função de Identificação do Cliente (key_func): uma função é definida para determinar quem está fazendo a requisição, permitindo rastrear o limite para cada cliente individualmente.

```
def get_client_identifier(request: Request) -> str:

    # Prioriza o nome de usuário (se autenticado); caso contrário, usa o IP

    return username or ip
```

Inicialização do Limiter: o objeto Limiter é configurado com a função de identificação e limites padrão.

```
limiter = Limiter(

    key_func=get_client_identifier, # Como identificar cada cliente

    default_limits=["60/minute"] # Limite padrão para todas as rotas

)
```

Aplicação em Endpoints: o limite é aplicado a rotas específicas através de um decorador.

```
@app.post("/predict")

    @limiter.limit("30/minute") # Limita a 30 requisições por minuto para este endpoint

    def predict(...):

        ...
```

Os limites de requisição são configurados de forma estratégica, levando em conta a necessidade de proteção e o recurso específico de cada endpoint.

ENDPOINT	LIMITE CONFIGURADO	MOTIVO
/login	10/min	Prevenção contra ataques de força bruta.
/predict	30/min	Proteção dos recursos de Machine Learning (CPU/Memória).
/predict/batch	10/min	Batches são operações pesadas e devem ser limitadas.
Demais	60/min	Limite padrão razoável para operações gerais.

Tabela 5 - Limites de requisição
Fonte: Elaborado pelo autor (2026)

Vamos modularizar também nossa API e separar os arquivos. Dentro da pasta app vamos criar o rate_limit.py:

```
"""
Rate Limiting com SlowAPI
Protege a API contra abuso e ataques DDoS

Rate Limiting = limitar quantidade de requisicoes por tempo
Exemplo: 30 requisicoes por minuto por IP

Por que usar:

SlowAPI eh baseado no Flask-Limiter, adaptado para FastAPI
"""
import os

from slowapi import Limiter
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded
from fastapi import Request
```

```

from fastapi.responses import JSONResponse

from app.logging_config import logger
from app.metrics import RATE_LIMIT_EXCEEDED

# Funcao que identifica o cliente (por IP)
# Em producao, pode usar header X-Forwarded-For se estiver
atras de proxy
def get_client_identifier(request: Request) -> str:
    """
    Retorna identificador unico do cliente para rate
    limiting.

    Usa o IP do cliente. Em producao atras de load balancer,
    considere usar X-Forwarded-For header.
    """
    # Se estiver atras de proxy (Render, AWS, etc), pega IP
    forwarded = request.headers.get("X-Forwarded-For")
    if forwarded:
        return forwarded.split(",")[0].strip()
    return get_remote_address(request)

# Limites padrão (podem ser sobrescritos por endpoint)
# Formato: "X per Y" onde Y pode ser: second, minute, hour,
day
DEFAULT_RATE_LIMIT = os.getenv("RATE_LIMIT_DEFAULT",
"60/minute")
PREDICT_RATE_LIMIT = os.getenv("RATE_LIMIT_PREDICT",
"30/minute")
BATCH_RATE_LIMIT = os.getenv("RATE_LIMIT_BATCH",
"10/minute")
LOGIN_RATE_LIMIT = os.getenv("RATE_LIMIT_LOGIN",
"10/minute")

# Cria o limiter
limiter = Limiter(
    key_func=get_client_identifier,
    default_limits=[DEFAULT_RATE_LIMIT],
    storage_uri="memory://", # Em producao, use Redis para
multiplicas instancias
)
=====
# HANDLER DE ERRO CUSTOMIZADO
#
=====
=====

```

```
    async def rate_limit_exceeded_handler(request: Request,
exc: RateLimitExceeded):
    """
    Handler customizado quando rate limit eh excedido.

    Retorna JSON amigavel em vez de texto puro.
    Tambem loga e incrementa metrica Prometheus.
    """
    client_ip = get_client_identifier(request)
    endpoint = request.url.path

    # Log estruturado
    logger.warning(
        "rate_limit_exceeded",
        extra={
            "client_ip": client_ip,
            "endpoint": endpoint,
            "limit": str(exc.detail),
        }
    )

    # Metrica Prometheus
    RATE_LIMIT_EXCEEDED.labels(endpoint=endpoint,
client_ip=client_ip).inc()

    return JSONResponse(
        status_code=429,
        content={
            "error": "rate_limit_exceeded",
            "message": f"Muitas requisicoes. Limite:
{exc.detail}",
            "retry_after_seconds": 60, # Sugestao de
quando tentar novamente
        },
        headers={"Retry-After": "60"}
    )
```

Código-fonte 1 – rate_limit.py
Fonte: Elaborado pelo autor (2026)

Desafio 2 - Adoção de Logs Estruturados (formato JSON)

A chave para um monitoramento eficiente de APIs reside na estrutura dos logs, permitindo uma análise rápida e detalhada do comportamento do sistema.

Logs em formato de texto simples, como os exemplos a seguir, apresentam sérias limitações para a observabilidade:

2024-01-15 10:30:45 INFO Login bem-sucedido: admin.

2024-01-15 10:30:46 ERROR Erro na predicacao: modelo nao carregado.

2024-01-15 10:30:47 INFO Predicacao: setosa para user admin.

Dificuldades

- **Filtro Complexo:** é difícil isolar eventos específicos (e.g., todos os erros de um usuário).
- **Agregação Limitada:** não é simples calcular métricas (e.g., latência média por endpoint).
- **Correlação Ausente:** não há um meio fácil de rastrear a sequência de eventos de uma única requisição.

A adoção de logs em formato JSON transforma os dados brutos em informações ricas e pesquisáveis:

```
{  
  "timestamp": "2024-01-15T10:30:45.123Z",  
  "level": "INFO",  
  "service": "api-iris-v2",  
  "trace_id": "abc12345",  
  "message": "prediction_completed",  
  "user": "admin",  
  "classe": "setosa",  
  "confianca": 0.98,  
  "latency_ms": 12.5  
}
```

Vantagens do Log Estruturado

- **Filtro Eficaz:** use campos nomeados, como jq 'select(.user == "admin")', para buscas precisas.
- **Agregação Poderosa:** utilize consultas baseadas em campos para métricas: SELECT AVG(latency_ms) FROM logs.
- **Correlação Simples:** o uso do trace_id permite rastrear todo o fluxo de uma requisição: WHERE trace_id = "abc12345".

CAMPO	DESCRIÇÃO	EXEMPLO
timestamp	Momento do evento (ISO 8601)	2024-01-15T10:30:45.123Z
level	Nível de severidade	DEBUG, INFO, WARNING, ERROR
service	Identificador do serviço/aplicação	api-iris-v2
trace_id	ID único da requisição	abc12345

message	Descrição do tipo de evento	prediction_completed
user	Identificador do usuário	admin
latency_ms	Tempo de resposta/processamento	12.5

Tabela 6 – Campos Essenciais para um Log Útil
Fonte: Elaborado pelo autor (2026)

O campo `trace_id` é fundamental, pois permite a correlação de múltiplos eventos de log que compõem o ciclo de vida de uma única requisição, mesmo que ela passe por diversos componentes do sistema.

Exemplo de Jornada de Requisição

Uma requisição `POST /predict` com `trace_id: abc12345` gera uma sequência de logs.

Exemplo de Rastreamento de uma Requisição (Request Tracing):

Requisição: `POST /predict`

ID de Rastreio (`trace_id`): `abc12345`

Fluxo de Logs Associados:

- **Início da Autenticação:** `{"trace_id": "abc12345", "message": "auth_start"}`.
- **Sucesso da Autenticação:** `{"trace_id": "abc12345", "message": "auth_success"}`.
- **Início da Predição:** `{"trace_id": "abc12345", "message": "predict_start"}`.
- **Conclusão da Predição:** `{"trace_id": "abc12345", "message": "predict_done"}`.

Buscar todos com: `WHERE trace_id = "abc12345"`.

Resultado: Rastreamento completo da jornada da requisição!

Vamos criar o arquivo `logging_config.py` na nossa pasta APP:

```

"""
Configuracao de Logs Estruturados (JSON)
Facilita analise em ferramentas como CloudWatch, Kibana,
Loggly

Logs estruturados permitem:
- Filtrar por campos especificos (user, endpoint, status)
- Correlacionar eventos com trace_id
- Integrar com ferramentas de observabilidade
"""
import logging
import sys
from datetime import datetime
from pythonjsonlogger import jsonlogger

class CustomJsonFormatter(jsonlogger.JsonFormatter):
    """
    Formatter customizado que adiciona campos extras a cada
log.

Campos adicionados automaticamente:
- timestamp: Data/hora em formato ISO 8601
- level: Nivel do log (INFO, WARNING, ERROR)
- service: Nome do servico (api-iris-v2)
- logger: Nome do logger
    """

    def add_fields(self, log_record, record, message_dict):
        super().add_fields(log_record, record,
message_dict)

        # Campos padrao em todo log
        log_record['timestamp'] = \
datetime.utcnow().isoformat() + 'Z'
        log_record['level'] = record.levelname
        log_record['service'] = 'api-iris-v2'
        log_record['logger'] = record.name

        # Garante que message existe
        if not log_record.get('message'):
            log_record['message'] = record.getMessage()

def setup_logging(level: str = "INFO") -> logging.Logger:
    """

```

Configura e retorna um logger com formato JSON.

Args:

level: Nivel minimo de log (DEBUG, INFO, WARNING, ERROR)

Returns:

Logger configurado para JSON estruturado

Exemplo de uso:

```
logger = setup_logging("INFO")
logger.info("user_login", extra={"username":
"admin", "ip": "192.168.1.1"})
```

Saida:

```
{ "timestamp": "2025-12-11T10:30:00Z", "level":
"INFO", "service": "api-iris-v2",
  "message": "user_login", "username": "admin",
"ip": "192.168.1.1" }
```

```
logger = logging.getLogger("api")
logger.setLevel(getattr(logging, level.upper()))
```

```
# Handler para stdout (container-friendly)
# Containers capturam stdout/stderr automaticamente
handler = logging.StreamHandler(sys.stdout)
handler.setFormatter(CustomJsonFormatter(
    '%(timestamp)s %(level)s %(name)s %(message)s'
))
```

```
# Remove handlers anteriores (evita duplicacao)
logger.handlers = []
logger.addHandler(handler)
logger.propagate = False
```

```
return logger
```

```
# Logger global para importar em outros modulos
logger = setup_logging()
```

Código-fonte 2 – logging_config.py
Fonte: Elaborado pelo autor (2026)

Middleware de Logging

Vamos criar um módulo middleware.py que intercepta todas as requisições automaticamente:

```
"""
Middlewares para Logging e Metricas
Intercepta todas as requisicoes para logging automatico

Middleware = codigo que roda ANTES e DEPOIS de cada
requisicao
Permite:
- Medir tempo de resposta automaticamente
- Adicionar trace_id para rastreamento
- Logar todas as requisicoes sem modificar endpoints
"""
import time
import uuid
from fastapi import Request
from starlette.middleware.base import BaseHTTPMiddleware
from app.logging_config import logger

class LoggingMiddleware(BaseHTTPMiddleware):
    """
    Middleware que loga todas as requisicoes
    automaticamente.

    Adiciona a cada requisicao:
    - trace_id: ID unico (UUID) para rastrear a requisicao
em todos os logs
    - latency_ms: Tempo de resposta em milissegundos
    - Headers de resposta com trace_id e tempo

    Fluxo:
    1. Request chega
    2. Middleware gera trace_id e marca inicio
    3. Request eh processada pelo endpoint
    4. Middleware calcula latencia e loga
    5. Response eh retornada com headers extras
    """

    async def dispatch(self, request: Request, call_next):
        # Gera trace_id unico para rastreamento
        # Permite correlacionar logs da mesma requisicao
        trace_id = str(uuid.uuid4())[:8] # Primeiros 8
caracteres do UUID
        request.state.trace_id = trace_id

        # Captura tempo inicial
        start_time = time.perf_counter()

        # Processa a requisicao (chama o endpoint)
        response = await call_next(request)
```

```
1000
    # Calcula latencia
    latency_ms = (time.perf_counter() - start_time) *

    # Log estruturado da requisicao
    # Nao loga /metrics para evitar poluicao
    (Prometheus acessa a cada 15s)
    if request.url.path != "/metrics":
        logger.info(
            "request_completed",
            extra={
                "trace_id": trace_id,
                "method": request.method,
                "path": request.url.path,
                "status_code": response.status_code,
                "latency_ms": round(latency_ms, 2),
                "client_ip": request.client.host if
request.client else None,
            }
        )

    # Adiciona headers de rastreamento na resposta
    # Uteis para debug do cliente
    response.headers["X-Trace-ID"] = trace_id
    response.headers["X-Response-Time-Ms"] =
str(round(latency_ms, 2))

    return response
```

Código-fonte 3 – módulo middleware.py

Fonte: Elaborado pelo autor (2026)

Desafio 3 - Processamento Eficiente - Batch Prediction

Otimizar o processamento de múltiplas predições é crucial para a performance de APIs de Machine Learning.

O Problema: Latência com Múltiplas Requisições Individuais.

Cenário de Exemplo: E-commerce de Jardinagem.

A empresa precisa classificar 50.000 amostras de flores diariamente.

ABORDAGEM INEFICIENTE (LOOP DE REQUISIÇÕES)	DETALHES
<pre>for flor in flores_50000: response = requests.post("/predict", json=flor)</pre>	50.000 requisições HTTP
	50.000 roundtrips de rede
	50.000 autenticações JWT
Tempo Estimado	~4 HORAS
Custo de Servidor	ALTO

Tabela 7 – Abordagem Ineficiente e detalhes
Fonte: Elaborado pelo autor (2026)

A Solução: O Endpoint /predict/batch.

A estratégia batch agrupa todas as amostras em uma única requisição, eliminando o overhead repetitivo.

ABORDAGEM EFICIENTE (BATCH)	DETALHES
<pre>response = requests.post("/predict/batch", json={"samples": flores_50000})</pre>	1 requisição HTTP
	1 roundtrip de rede
	1 autenticação JWT
Tempo Estimado	~30 SEGUNDOS
Ganho de Performance	~480x mais rápido!

Tabela 8 – Estratégia batch
Fonte: Elaborado pelo autor (2026)

- **Por Que o Processamento Batch é Superior?**
 - A principal vantagem reside na minimização do overhead de comunicação (rede, autenticação etc.).
- **Comparativo de Overhead**
 - Requisição Individual:
 - ~50ms de overhead POR requisição.
 - Exemplo para 50.000 requisições: $50.000 \times 50\text{ms} = 41 \text{ minutos}$ (só de overhead).
 - Processamento Batch:
 - ~50ms de overhead UMA VEZ (para o lote completo).
 - Exemplo para 50.000 requisições (em um lote): $1 \times 50\text{ms} =$

50ms (overhead total).

Anatomia do Endpoint /predict/batch

Estrutura da Requisição (Request): o corpo da requisição deve ser um array de amostras dentro de uma chave, como "samples".

POST /predict/batch

Authorization: Bearer <token>

```
{
  "samples": [
    {"sepal_length": 5.1, "sepal_width": 3.5, "petal_length": 1.4, "petal_width": 0.2},
    {"sepal_length": 7.0, "sepal_width": 3.2, "petal_length": 4.7, "petal_width": 1.4},
    {"sepal_length": 6.3, "sepal_width": 3.3, "petal_length": 6.0, "petal_width": 2.5}
  ]
}
```

Estrutura da Resposta (Response): a resposta inclui metadados do processamento e o array de predições, mapeadas pelo seu índice na requisição.

```
{
  "sucesso": true,
  "total_processadas": 3,
  "tempo_total_ms": 12.5,
  "predicoes": [
    {"indice": 0, "classe": "setosa", "confianca": 0.98},
    {"indice": 1, "classe": "versicolor", "confianca": 0.95},
    {"indice": 2, "classe": "virginica", "confianca": 0.97}
  ]
}
```

É fundamental implementar limites de segurança para evitar sobrecarga de memória no servidor.

Limite máximo de amostras por requisição

MAX_BATCH_SIZE = 1000

if len(samples) > MAX_BATCH_SIZE:

```
    raise HTTPException(
        status_code=400,
        detail=f"Batch muito grande. Máximo: {MAX_BATCH_SIZE}"
    )
```

Vamos criar dentro do nosso módulo routers/predict.py a rota batch predict:

```
@router.post("/predict/batch",
response_model=BatchPredictResponse)
@limiter.limit(BATCH_RATE_LIMIT)
def predict_batch(
    request: Request,
    payload: BatchPredictRequest,
    current_user: dict = Depends(get_current_user),
):
    """
    Faz predicacao para multiplas flores Iris de uma vez.

    **Rate Limit:** 10 requisicoes por minuto

    **Maximo:** 100 flores por requisicao

    **Requer autenticacao:** Inclua o header
    `Authorization: Bearer <token>`

    **Vantagens do Batch:**
    - Mais eficiente que multiplas chamadas individuais
    - Menor overhead de rede
    - Ideal para processamento em massa
    """
    if not MODELO_OK:
        raise HTTPException(status_code=503,
        detail="Modelo nao disponivel")

    trace_id = getattr(request.state, "trace_id", "N/A")
    start = time.perf_counter()

    # Prepara features de todas as flores
    features_list = [
        [
            item.sepal_length,
            item.sepal_width,
            item.petal_length,
            item.petal_width,
        ]
        for item in payload.items
    ]
    features = np.array(features_list)

    # Predicao em lote (mais eficiente que loop)
    pred_indices = modelo.predict(features)
    all_probs = modelo.predict_proba(features)

    # Monta resposta
    predicoes = []
```

```
        for i, (pred_idx, probs) in enumerate(zip(pred_indices,
all_probs))):
            classe = classes[pred_idx]
            confidence = float(max(probs))

            predicoes.append(
                BatchPredictItem(
                    indice=i,
                    classe=classe,
                    confianca=round(confidence, 4),
                    probabilidades={
                        classes[j]: round(float(p), 4) for j,
p in enumerate(probs)
                    },
                )
            )

            # Metrica por classe
            PREDICTIONS_TOTAL.labels(classe=classe,
user=current_user["username"]).inc()

            latency = time.perf_counter() - start
            batch_size = len(payload.items)

            # Metricas
            BATCH_PREDICTIONS_TOTAL.labels(
                user=current_user["username"],
batch_size=str(batch_size)
            ).inc()
            BATCH_PREDICTION_LATENCY.observe(latency)

            # Log
            logger.info(
                "batch_prediction_completed",
                extra={
                    "trace_id": trace_id,
                    "user": current_user["username"],
                    "batch_size": batch_size,
                    "latency_ms": round(latency * 1000, 2),
                    "avg_latency_per_item_ms": round((latency *
1000) / batch_size, 2),
                },
            )

            return BatchPredictResponse(
                sucesso=True,
                total=batch_size,
                tempo_total_ms=round(latency * 1000, 2),
                predicoes=predicoes,
                usuario=current_user["username"],
            )
```

Código-fonte 4 – Rota batch predict
Fonte: Elaborado pelo autor (2026)

Esse é um bom momento para olharmos para as boas práticas do nosso código.

Boas práticas - Modularizando o main.py e a documentação!

O arquivo main.py da v2 tornou-se excessivamente complexo. O problema é que ele centralizava diversas responsabilidades, como configuração da aplicação, definição de schemas Pydantic, carregamento do modelo e definição de rotas.

Em projetos reais, essa centralização é um grande obstáculo, dificultando a localização do código, a evolução de novas funcionalidades e a criação de testes.

Benefícios da Modularização

A modularização resolve esses problemas e traz as seguintes vantagens:

- **Separação de Responsabilidades (SRP):** garante que cada arquivo tenha um propósito único e bem definido.
- **Melhoria na Leitura e Manutenção:** o main.py fica reduzido apenas à função de bootstrap (inicialização) da API.
- **Reuso e Testabilidade:** schemas, loaders e routers podem ser importados e testados de forma independente.
- **Escalabilidade:** permite adicionar novas rotas ou versões da API sem sobrecarregar um único arquivo.

No final, assim ficou a estrutura do nosso projeto:

Estrutura do Projeto

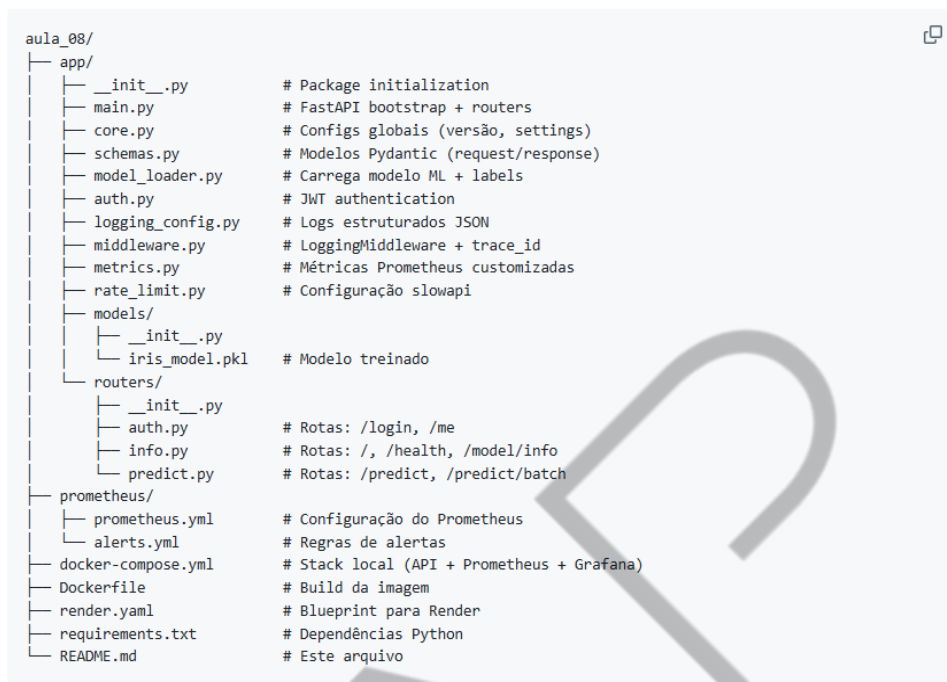


Figura 1 - Estrutura do Projeto
Fonte: Elaborado pelo autor (2025)

Aqui fica a dica: use copilot ou codex para te ajudarem na documentação Readme do seu projeto; isso vai acelerar em muito sua produtividade, desde que você esteja sempre validando as informações que a IA te ajudou a elaborar!

Aqui você encontra o [link do repositório](#) para que seja possível visualizar a documentação e clonar ele para reproduzir no Render.

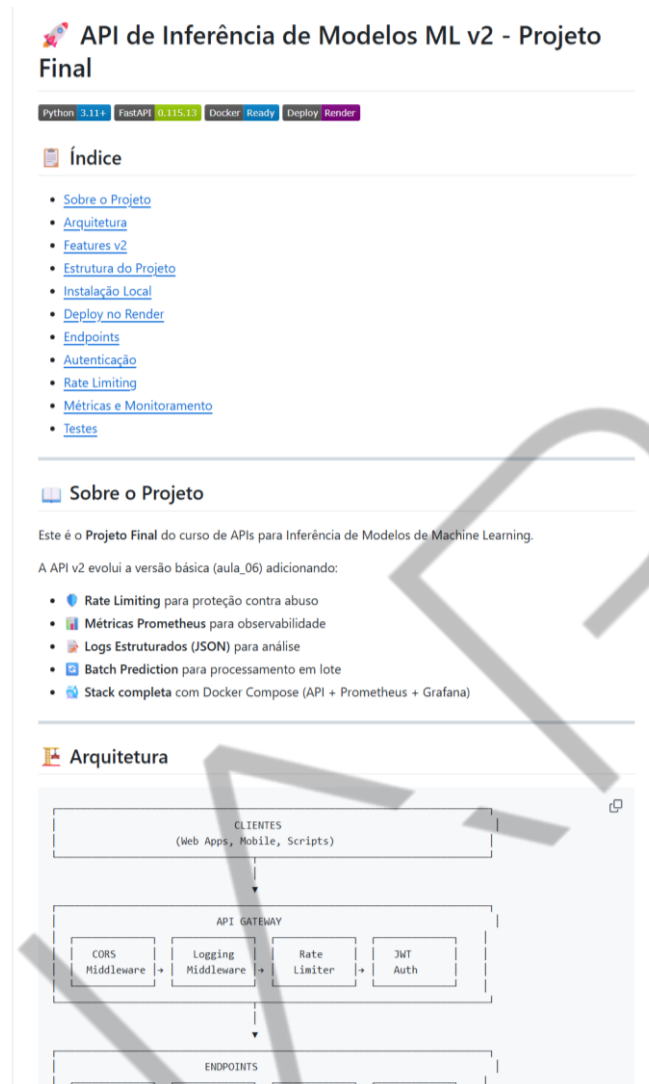


Figura 2 – Readme
Fonte: Elaborado pelo autor (2025)

Desafios 4 e 5 - Deploy no Render e métricas!

Depois de clonar o repositório, vamos subir os containers com o Prometheus e Grafana no render.

No local, usamos o docker-compose para buildar todos serviços (deixei essa opção no repositório para aplicarmos os testes locais). Poderíamos subir um webservice por vez com seu respectivo dockerfile, mas o render permite que utilizemos um arquivo render.yaml como blueprint para que a plataforma nos auxilie no deploy!

```
#
=====
# RENDER BLUEPRINT - API Iris v2 (Projeto Final)
#
=====
# Este arquivo define a infraestrutura para deploy no
Render.
#
# Diferenças da v1 (aula_06):
# - Nome do serviço: api-iris-v2 (para coexistir com v1)
# - Novas variáveis de ambiente para rate limiting
# - Mesmo modelo, mais features de produção
#
# Para deploy:
# 1. Conecte o repositório ao Render
# 2. Selecione este arquivo como Blueprint
# 3. O Render criará o serviço automaticamente
#
=====
services:
#
=====
# WEB SERVICE - API de Inferência v2
#
=====
- type: web
  name: api-iris-v2
  runtime: docker
  region: ohio # Mesma região da v1 para consistência
#
=====
# Plano de recursos
#
=====
# free: Ideal para desenvolvimento/aprendizado
# starter: Para produção leve ($7/mês)
# standard: Para produção com mais recursos ($25/mês)
plan: free
#
=====
# Configuração do Docker
```

```
#
=====
=====
    dockerfilePath: ./Dockerfile
    dockerContext: .

#
=====
=====
    # Health Check
    #
=====
=====
    # O Render verifica periodicamente se a API está
    respondendo
    healthCheckPath: /health
    #
=====
=====
    # Variáveis de Ambiente
    #
=====
=====
    envVars:
        # -----
        -----
        # Configuração da Aplicação
        # -----
        -----
        - key: APP_ENV
          value: production

        - key: LOG_LEVEL
          value: INFO
        # -----
        -----
        # Segurança - JWT
        # -----
        -----
        # IMPORTANTE: Em produção, gere uma chave secreta
        forte!
        # Use: python -c "import secrets;
        print(secrets.token_hex(32))"
        - key: SECRET_KEY
          generateValue: true # Render gera automaticamente

        - key: ALGORITHM
          value: HS256

        - key: ACCESS_TOKEN_EXPIRE_MINUTES
          value: "30"
```

```
# -----  
-----  
# Rate Limiting  
# -----  
-----  
# Configurações de limite de requisições por minuto  
- key: RATE_LIMIT_DEFAULT  
  value: "60" # Requisições padrão por minuto  
  
- key: RATE_LIMIT_PREDICT  
  value: "30" # Predições individuais por minuto  
  
- key: RATE_LIMIT_BATCH  
  value: "10" # Predições em batch por minuto  
  
- key: RATE_LIMIT_LOGIN  
  value: "10" # Tentativas de login por minuto  
# -----  
-----  
# Configuração do Modelo  
# -----  
-----  
- key: MODEL_PATH  
  value: "app/models/iris_model.pkl"  
  
- key: MODEL_VERSION  
  value: "2.0.0"  
# -----  
-----  
# Métricas (desabilitado em produção free tier)  
# -----  
-----  
# No plano free do Render, não temos  
Prometheus/Grafana  
# As métricas ficam expostas em /metrics mas não são  
coletadas  
- key: METRICS_ENABLED  
  value: "true"  
  
# -----  
-----  
# CORS  
# -----  
-----  
# Em produção, configure os origins permitidos  
# Exemplo:  
"https://meusite.com,https://app.meusite.com"  
- key: CORS_ORIGINS  
  value: "*" # Permite todos (apenas para demo)  
# -----  
-----
```

```
# Usuários de demo
# -----
-----
# Em produção real, use um banco de dados!
- key: DEMO_USER
  value: "admin"

- key: DEMO_PASSWORD
  value: "secret123"

#
=====
=====
# WEB SERVICE - Prometheus (UI + scrape de métricas)
#
=====
=====
- type: web
  name: prometheus-iris-v2
  runtime: docker
  region: ohio
  plan: free
  dockerContext: ./prometheus
  dockerfilePath: ./Dockerfile
  envVars:
    # Aponta para a API pública (Render dá uma URL por
serviço)
    - key: API_TARGET
      value: "api-iris-v2.onrender.com"
    - key: API_SCHEME
      value: "https"
    - key: SCRAPE_INTERVAL
      value: "15s"

#
=====
=====
# WEB SERVICE - Grafana (UI de dashboards)
#
=====
=====
- type: web
  name: grafana-iris-v2
  runtime: docker
  region: ohio
  plan: free
  dockerContext: ./grafana
  dockerfilePath: ./Dockerfile
  envVars:
    - key: GF_SECURITY_ADMIN_USER
      value: "admin"
    - key: GF_SECURITY_ADMIN_PASSWORD
```

```

      value: "admin"
    - key: GF_USERS_ALLOW_SIGN_UP
      value: "false"
    # Datasource provisionado em
grafana/provisioning/datasources/prometheus.yml
    - key: PROMETHEUS_URL
      value: "https://prometheus-iris-v2.onrender.com"

```

Código-fonte 5 – render.yaml como blueprint
Fonte: Elaborado pelo autor (2026)

No render, vamos criar uma “New Blueprint” e pedir para que ele crie os webservices:

You are deploying from a render.yaml for **iohpedro/API-IRIS-RENDER**

Blueprint Name
A unique name for your Blueprint.
api-iris-prod

Branch
The repository branch with the `render.yaml` file.
main

Review render.yaml configurations from iohpedro/API-IRIS-RENDER.

All future updates to `render.yaml` will be synced automatically, which may change your costs.

Specified configurations
We found existing services that match this Blueprint.
☒ Associate existing services
☐ Create all as new services

- Create web service **grafana-iris-v2**
- Associate web service **prometheus-iris-v2** with this Blueprint
- Update web service **prometheus-iris-v2** root directory to `<empty>`
- Update web service **prometheus-iris-v2** static publish path to `<empty>`
- Update web service **prometheus-iris-v2** Dockerfile path to `./prometheus/Dockerfile`
- Update web service **prometheus-iris-v2** base directory to `./prometheus`
- Update environment variable with key **API_TARGET** for web service **prometheus-iris-v2**
- Update environment variable with key **SCRAPE_INTERVAL** for web service **prometheus-iris-v2**
- Associate web service **api-iris-v2** with this Blueprint
- Update web service **api-iris-v2** root directory to `<empty>`
- Update web service **api-iris-v2** health check path to `/health`
- Update web service **api-iris-v2** static publish path to `<empty>`
- Create environment variable with key **APP_ENV** for web service **api-iris-v2**
- Create environment variable with key **LOG_LEVEL** for web service **api-iris-v2**
- Create environment variable with key **SECRET_KEY** for web service **api-iris-v2**
- Create environment variable with key **ALGORITHM** for web service **api-iris-v2**
- Create environment variable with key **ACCESS_TOKEN_EXPIRE_MINUTES** for web service **api-iris-v2**
- Create environment variable with key **RATE_LIMIT_DEFAULT** for web service **api-iris-v2**
- Create environment variable with key **RATE_LIMIT_PREDICT** for web service **api-iris-v2**
- Create environment variable with key **RATE_LIMIT_BATCH** for web service **api-iris-v2**
- Create environment variable with key **RATE_LIMIT_LOGIN** for web service **api-iris-v2**
- Create environment variable with key **MODEL_PATH** for web service **api-iris-v2**

Figura 3 - Render.yaml
Fonte: Elaborado pelo autor (2025)

Specified configurations
We found existing services that match this Blueprint, but those services are already associated with an existing Blueprint instance.
☐ Associate existing services
☒ Create all as new services

- Create web service **api-iris-v2-d20g**
- Create web service **prometheus-iris-v2-d20g**
- Create web service **grafana-iris-v2-d20g**

Deploy Blueprint **Cancel**

Figura 4 - Render.yaml – webservices
Fonte: Elaborado pelo autor (2025)

O render cria cada um dos serviços que estão agora sustentando a nossa API:

SERVICE NAME	STATUS	RUNTIME	REGION	UPDATED
prometheus-iris-v2-d20g	✓ Deployed	Docker	Oregon	4min
api-iris-v2-d20g	✓ Deployed	Docker	Oregon	6min
grafana-iris-v2-d20g	✓ Deployed	Docker	Oregon	7min

Figura 5 - Render.yaml - webservices deployed
Fonte: Elaborado pelo autor (2025)

Vamos agora testar nossa API em produção!

[API em produção.](#)

[Métricas Prometheus.](#)

[Dashboards com Grafana.](#)

Walkthrough e considerações finais!

Agora temos uma API robusta em produção. Nela, podemos ver na rota /redoc, por exemplo, a documentação do nosso novo recurso predição em batch antes de testarmos:

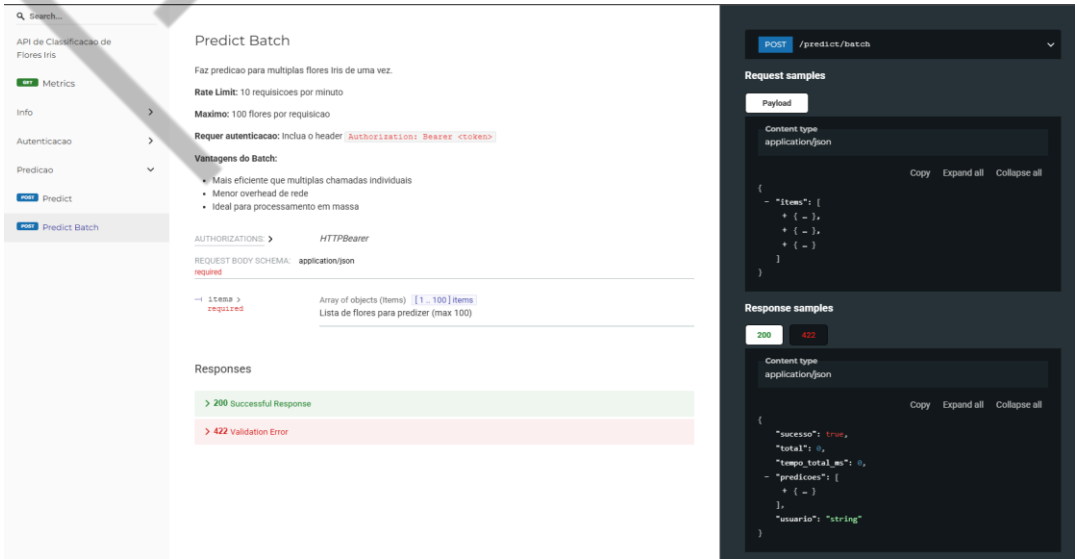


Figura 6 - Redoc - Nova Rota
Fonte: Elaborado pelo autor (2025)

Agora vamos testar no swagger:

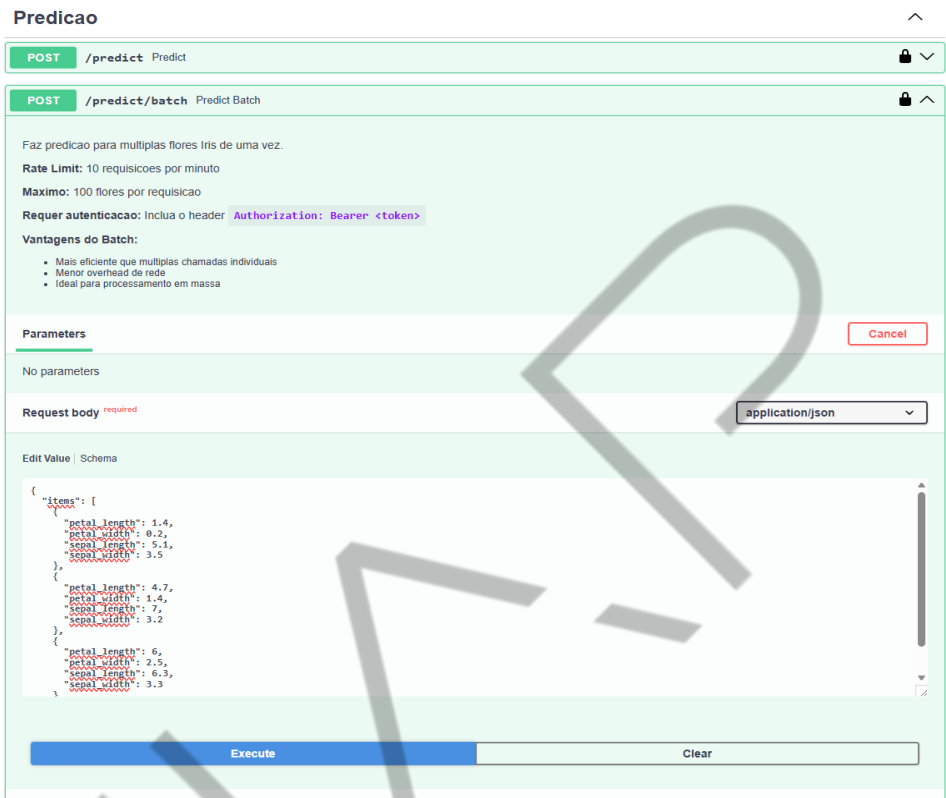


Figura 7 - Swagger - Testando Nova Rota
Fonte: Elaborado pelo autor (2025)

Funcionou! Temos nosso endpoint fazendo uma predição em batch!

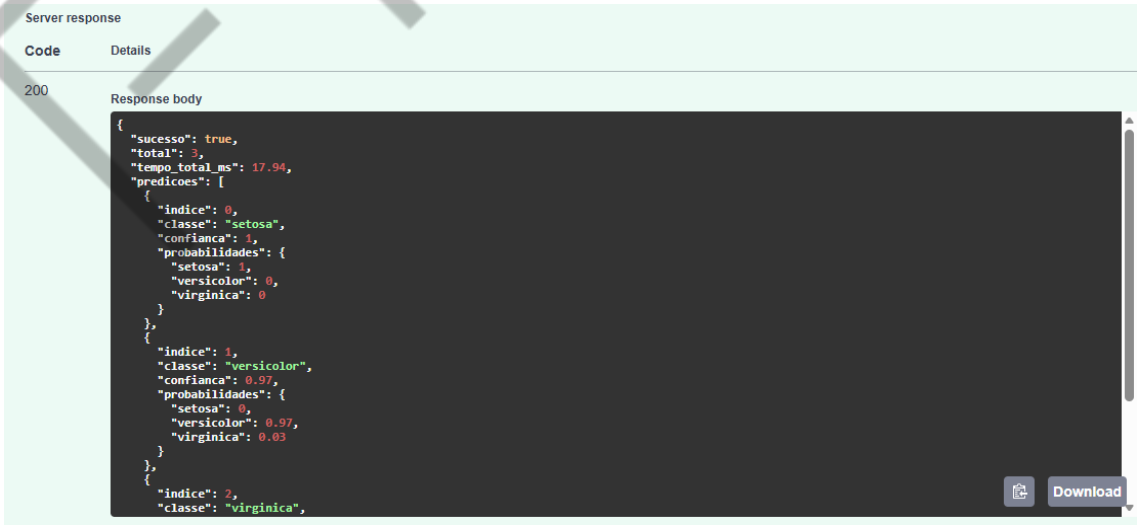


Figura 8 - Swagger - Predição em Batch!
Fonte: Elaborado pelo autor (2025)

Também podemos [acessar](#) o Prometheus e o Grafana.

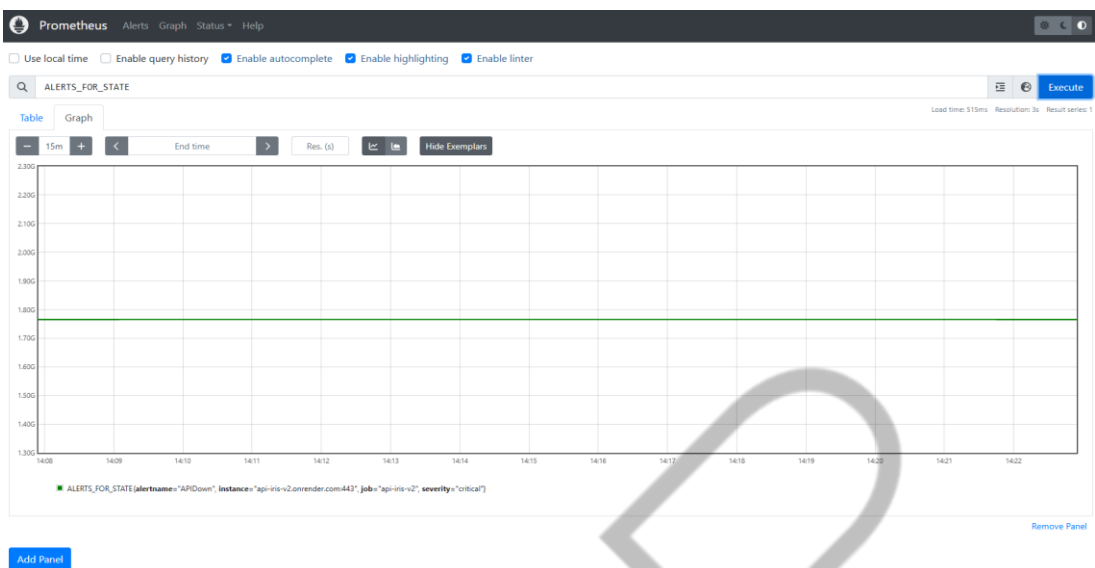


Figura 9 - Prometheus - Gráfico ALERTS
Fonte: Elaborado pelo autor (2025)

Podemos identificar um alerta quando a API fica offline. No nosso caso isso é normal, pois o render deixa o webservice desligado por ociosidade na versão gratuita.

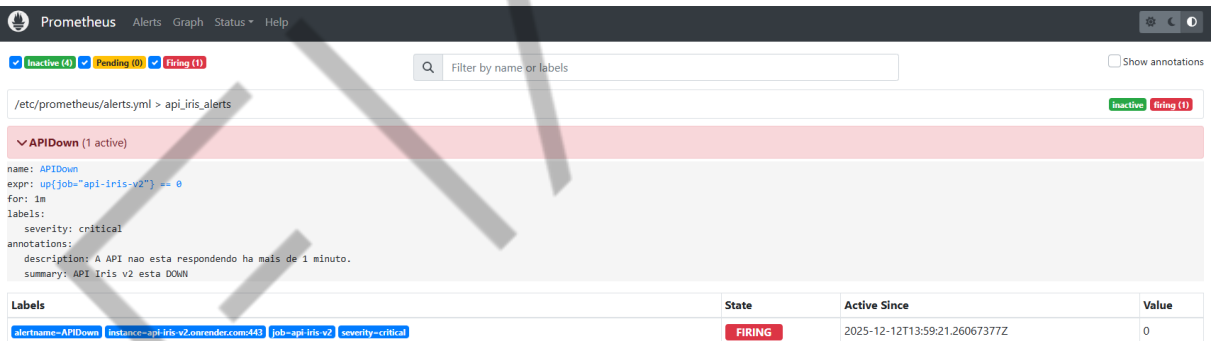


Figura 10 - Prometheus - Visão dos Alertas
Fonte: Elaborado pelo autor (2025)

Vamos [acessar](#) agora o Grafana e criar um dashboard.

Nas conexões, vamos selecionar nosso data source Prometheus disponibilizando a URL do serviço:

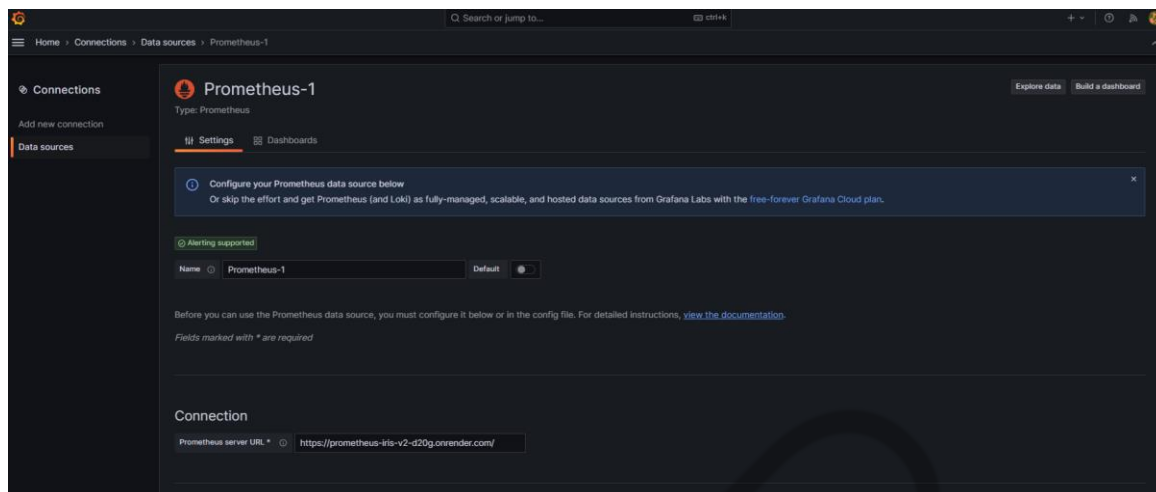


Figura 11 - Grafana - Conexão Prometheus
Fonte: Elaborado pelo autor (2025)

Podemos agora criar uma visualização dos nossos alertas e salvar em um dashboard:

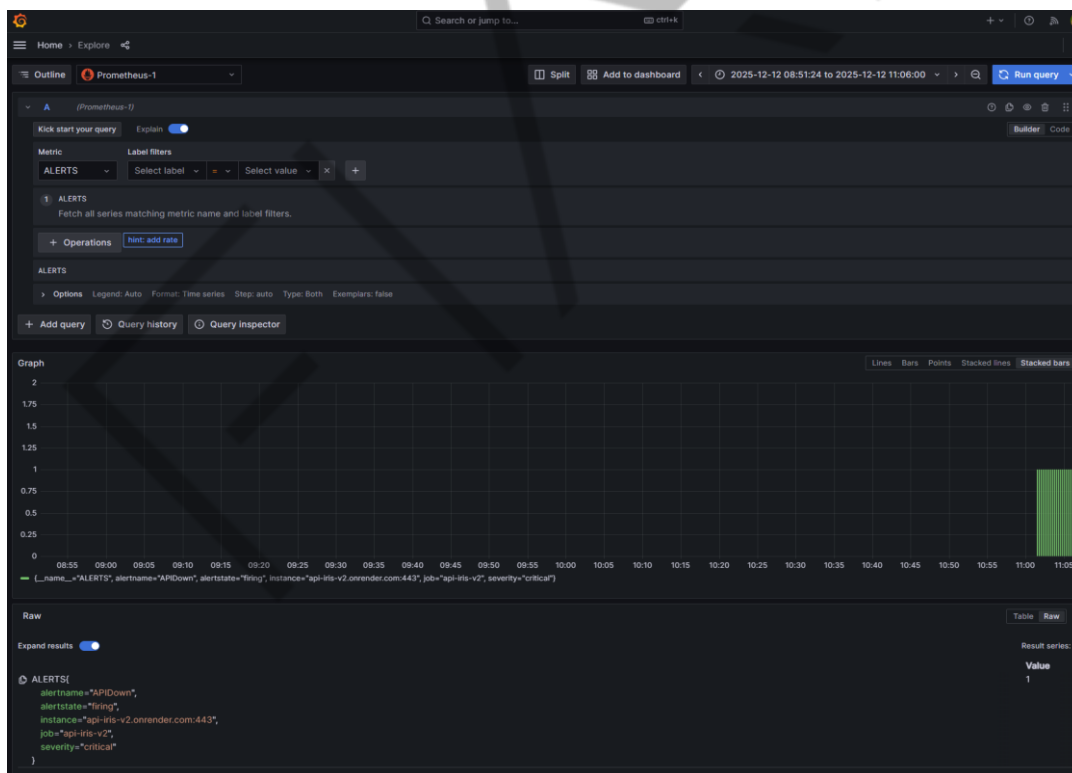


Figura 12 - Grafana - Criando visualização
Fonte: Elaborado pelo autor (2025)

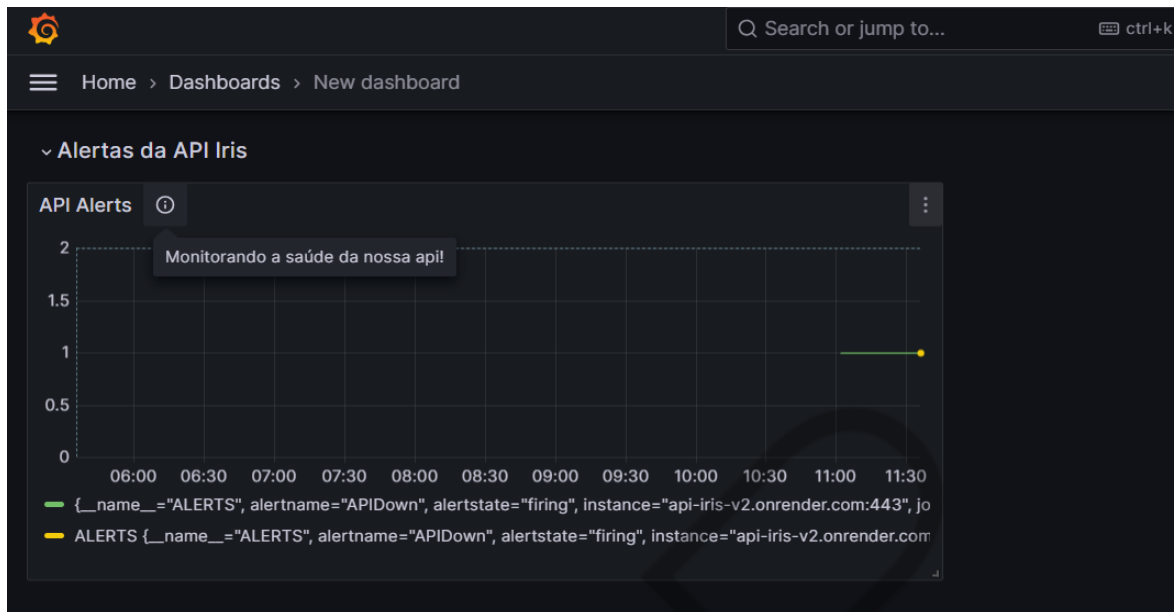


Figura 13 - Grafana - Salvando Dashboard!
Fonte: Elaborado pelo autor (2025)

Parabéns! Você agora sabe:

- Construir APIs REST com FastAPI.
- Proteger APIs com JWT e Rate Limiting.
- Containerizar com Docker.
- Fazer deploy em nuvem (Render).
- Monitorar com Logs + Prometheus + Grafana.
- Processar em batch para eficiência.
- Versionar e evoluir APIs de forma profissional.

Agora é a sua vez de aplicar tudo isso em projetos reais: teste acrescentar novas rotas a nossa API ou criar diversas visualizações nos dashboards do Grafana e, principalmente, se aprofunde nos materiais de referência, além de dominar a construção de APIs para inferência de modelos de Machine Learning!

O QUE VOCÊ VIU NESTA AULA?

Nesta aula final, você pegou a nossa API Iris e fechou o ciclo completo de engenharia para produção, publicando a versão v2 com tudo o que construímos nas aulas anteriores. Integramos JWT, rate limiting, logs estruturados, métricas e adicionamos um novo endpoint de predição em batch, transformando a API em um serviço mais eficiente, seguro e pronto para uso real.

Na parte prática, você viu como organizar o projeto de forma profissional, modularizando o código (separando responsabilidades como logging, métricas e rate limiting) e como fazer o deploy no Render usando blueprint (render.yaml) para subir não só a API, mas também o ecossistema de observabilidade com Prometheus e Grafana, validando o funcionamento via Swagger/Redoc e testando a API já em produção.

Por fim, você consolidou a visão de operação: monitorar o serviço rodando, entender alertas e limitações do ambiente (como o sleep do plano free) e enxergar que publicar uma API de ML não é “só dar deploy”, é entregar um produto técnico com contrato, segurança, performance, observabilidade e capacidade de evolução para virar base de projetos futuros.

REFERÊNCIAS

GÉRÓN, A. **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow**. Beijing: O'Reilly Media, 2023.

GIFT, N.; DEZA, A. **Practical MLOps**: Operationalizing Machine Learning Models. Sebastopol: O'Reilly Media, 2021.

LUBANOVIC, B. **FastAPI**: Modern Python Web Development. Sebastopol: O'Reilly Media, 2023.

NEWMAN, S. **Building Microservices**: Designing Fine-Grained Systems. Sebastopol: O'Reilly Media, 2021.

PALAVRAS-CHAVE

JWT. Token. API. WEB HTTP. REST. API RESTful. GraphQL. gRPC. JSON. MLOps.

FINAL

EMSE